

PREMIER UNIVERSITY

Department of Computer Science & Engineering



Final Lab Report

Course Name: Big Data Analytics Laboratory.

Course code: CSE - 4346.

Name of Report: Movie Data Analysis Using Hadoop MapReduce and Apache Pig.

Date of Submission: 13-09-2026.

SUBMITTED TO,

MD Tamim Hossain

Lecturer,,
Computer Science and Engineering,
Premier University, Chittagong..

Remarks:

Submitted By:

Name	Mohammed Walid
ID	0222220005101147
Section	B
Semester	8th
Session	Spring - 2026

Table of Contents

1. Introduction.....	1
2. Problem Statement and Objectives.....	1
2.1 Problem Statement.....	1
2.2 Objectives.....	1
3. Dataset Description.....	1
4. Technologies and Hadoop Environment.....	2
5. Overall Data Processing Workflow.....	2
6. MapReduce Analysis.....	3
6.1 Top 10 Highest Grossing Movies.....	3
6.2 Total Global Box Office by Genre.....	6
7. Pig Latin Analysis.....	9
7.1 Top 10 Global Revenue Movies.....	9
7.2 Movies with Positive ROI.....	11
8. Comparative Analysis: MapReduce vs Pig.....	13
9. Results and Discussion.....	13
10. Limitations and Learning Outcomes.....	14
10.1 Limitations.....	14
10.2 Learning Outcomes.....	14
11. Conclusion.....	14

1. Introduction

The rapid growth of movie, revenue and audience-related data creates a need for scalable data-processing techniques. This project applies Hadoop-based technologies to a large movie dataset to extract useful information from revenue, genre, budget and profitability fields.

Two complementary approaches are used. Java MapReduce provides explicit control over Mapper and Reducer processing, while Apache Pig provides high-level data-flow operations for concise analysis. The project focuses on global revenue, genre-level revenue and movie profitability.

2. Problem Statement and Objectives

2.1 Problem Statement

A large movie dataset contains many individual records with financial and descriptive attributes. Manual processing is inefficient and does not demonstrate distributed data-processing concepts. The objective is therefore to process the dataset using Hadoop MapReduce and Apache Pig and convert raw movie records into meaningful analytical results.

2.2 Objectives

- Store and process the movie dataset in a Hadoop/HDFS environment.
- Identify the ten movies with the highest global box-office revenue using Java MapReduce.
- Calculate total global box-office revenue for each genre using Java MapReduce.
- Perform a Top-10 global-revenue analysis using Apache Pig.
- Calculate ROI and retain movies with ROI greater than zero using Pig.
- Demonstrate mapping, filtering, shuffling, grouping, reducing, sorting, limiting and output storage.
- Compare Java MapReduce and Pig Latin in terms of control, complexity and suitability.

3. Dataset Description

The supplied movie dataset contains 999,999 data records and 17 original columns. It covers 8 genres and 10 countries, with release years ranging from 1950 to 2025. The supplied project material reports no missing values in the original CSV columns.

Field	Description
MovieID	Unique movie identifier
Title	Movie title
Genre	Movie genre/category
ReleaseYear	Release year
ReleaseDate	Movie release date
Country	Associated country
BudgetUSD	Production budget in US dollars

US_BoxOfficeUSD	US box-office revenue
Global_BoxOfficeUSD	Worldwide/global box-office revenue
Opening_Day_SalesUSD	Opening-day sales
One_Week_SalesUSD	First-week sales
IMDbRating	IMDb rating
RottenTomatoesScore	Rotten Tomatoes score
NumVotesIMDb	Number of IMDb votes
NumVotesRT	Number of Rotten Tomatoes votes
Director	Director name
LeadActor	Lead actor name

Table 1: Movie Dataset Fields and Descriptions

The main fields used are Global_BoxOfficeUSD for revenue analysis, Genre for grouping, BudgetUSD for ROI calculation and MovieID/Title for identifying movie records.

4. Technologies and Hadoop Environment

Technology	Purpose
Hadoop	Distributed data-processing framework
HDFS	Distributed storage for input/output
MapReduce	Custom distributed processing in Java
YARN	Resource and job management
Apache Pig	High-level data-flow analysis
Java	MapReduce implementation
Maven	Build and dependency management

Table 2: Technologies Used and Their Purposes

The Java MapReduce programs were developed as a Maven project and packaged into a JAR before Hadoop execution. The movie CSV was uploaded to HDFS under /movies for the Hadoop MapReduce experiments.

5. Overall Data Processing Workflow

MapReduce workflow:



Figure 1: Hadoop MapReduce Data Processing Workflow

Pig workflow:

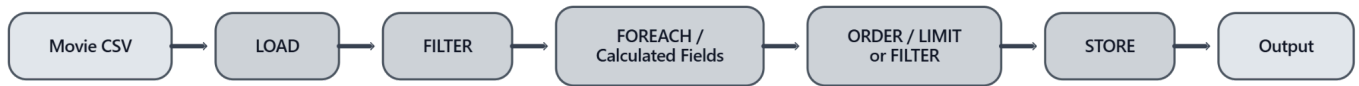


Figure 2: Apache Pig Data Processing Workflow

MapReduce provides direct control over record processing and data structures, whereas Pig provides high-level operators that directly represent common transformations.

6. MapReduce Analysis

Two Java MapReduce programs were developed: one for global Top-10 revenue ranking and one for total global box-office revenue by genre.

6.1 Top 10 Highest Grossing Movies

Objective

To identify the ten movies with the highest Global_BoxOfficeUSD values from the movie dataset.

Method

The Mapper extracts the movie title and global box-office revenue from each valid movie record and emits: TOP → Movie Title + Global Revenue.

All movies are assigned the same key, "TOP", so that they are grouped together and processed by a single Reducer. During the Shuffle and Sort phase, all movie records are therefore sent to the same Reducer. The Reducer maintains a PriorityQueue containing only the ten highest-revenue movies. If more than ten movies are stored, the movie with the smallest revenue is removed.

After all records are processed, the remaining ten movies are sorted in descending order of revenue.

Workflow

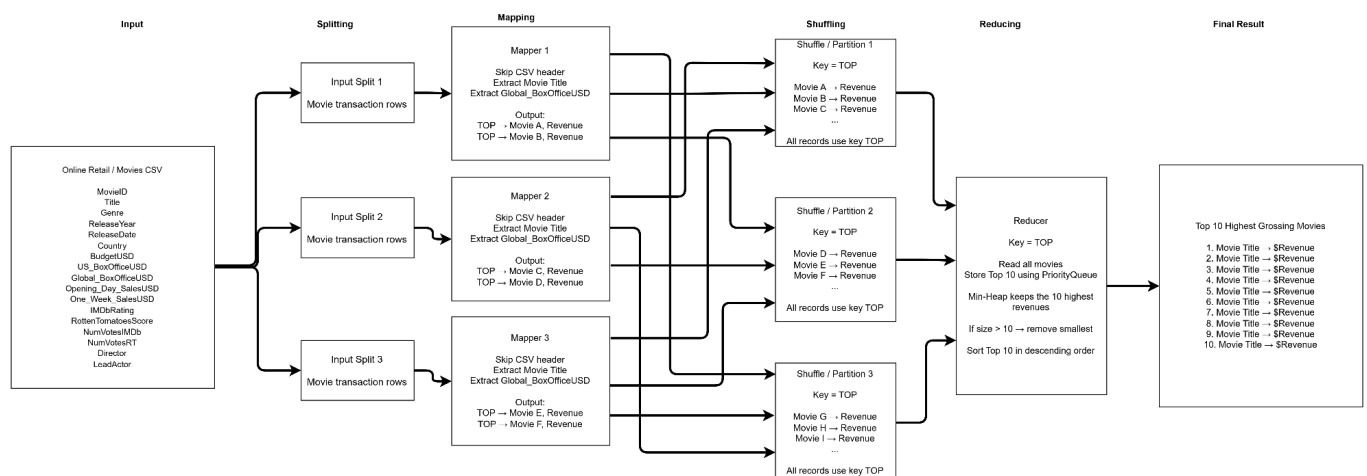


Figure 3: MapReduce Workflow for Top 10 Highest Grossing Movies

Mapper

The main Mapper logic is:

```
String movieTitle = fields[1].trim();

double globalBoxOffice =
Double.parseDouble(fields[8].trim());

outValue.set(
movieTitle + "\t" + globalBoxOffice
);

context.write(outKey, outValue);
```

The Mapper therefore converts each movie into a key-value pair: TOP → Movie Title + Global Revenue

Using the same "TOP" key ensures that all movies are sent to the same Reducer for a global ranking.

Reducer

The Reducer uses a PriorityQueue as a min-heap to maintain the current Top 10 movies:

```
top10.offer(movie);

if (top10.size() > 10) {
top10.poll();
}
```

When the queue contains more than ten movies, poll() removes the movie with the smallest revenue. Therefore, the queue always retains the ten highest-grossing movies processed so far.

After processing all movies, the remaining records are sorted in descending order:

```
java.util.Arrays.sort(
results,
(a, b) ->
Double.compare(
b.revenue,
a.revenue
));
```

Driver

The Driver configures and submits the MapReduce job:

```
Job job =
Job.getInstance(
conf,
"Top 10 Highest Grossing Movies"
);

job.setJarByClass(MovieTop10.class);

job.setMapperClass(MovieMapper.class);
job.setReducerClass(MovieReducer.class);

job.setMapOutputKeyClass(Text.class);
job.setMapOutputValueClass(Text.class);

job.setOutputKeyClass(Text.class);
job.setOutputValueClass(Text.class);

job.setNumReduceTasks(1);

FileInputFormat.addInputPath(job, new Path(args[0]));
FileOutputFormat.setOutputPath(job, new Path(args[1]));

System.exit(job.waitForCompletion(true) ? 0 : 1);
```

The Driver connects the input data with the Mapper and Reducer and specifies the input and output HDFS paths.

The program uses one Reducer because a single global Top 10 must be produced. Using one Reducer ensures that all movie records are considered together when determining the highest-grossing movies.

Result

Rank	Movie Title	Global Revenue (USD)
1	Scientist picture	1,499,496,720
2	Among religious night	1,499,467,818
3	Happy religious serious citizen	1,496,190,333
4	Free cup day tree	1,495,814,648
5	Cut	1,495,192,277
6	Seem fish send people	1,494,807,402
7	Production five rule	1,493,571,871
8	Which team deep	1,492,659,744

9	Serve view	1,489,685,999
10	Take exist good	1,487,863,886

Table 3: Top 10 Highest Grossing Movies Using MapReduce

Observation: The PriorityQueue keeps only ten candidates, demonstrating an efficient global Top-K strategy.

The figure consists of three screenshots of a Windows Command Prompt running Hadoop MapReduce commands. The first screenshot shows the setup of the Hadoop environment, including starting the NameNode and DataNode daemons, and creating a directory for the movies dataset. The second screenshot shows the execution of the MapReduce job, displaying various log messages and progress indicators. The third screenshot shows the final output of the job, listing the top 10 highest grossing movies and their revenue.

```

Administrator: Command Prompt
C:\>hadoop\bin>start-all.cmd
This script is deprecated. Instead use start-dfs.cmd and start-yarn.cmd
starting yarn daemons
C:\>hadoop\bin>jps
3520 ResourceManager
12504 NodeManager
4280 DataNode
10620 jps
9788 NameNode
C:\>hadoop\bin>hdfs dfs -mkdir /movies
C:\>hadoop\bin>hdfs dfs -put "C:\Users\Walid\Desktop\bda\movies_dataset.csv" /movies/
C:\>hadoop\bin>hdfs dfs -ls /
Found 1 items
drwxr-xr-x - Walid supergroup 0 2026-09-11 16:19 /movies
C:\>hadoop\bin>hdfs dfs -ls /movies
Found 1 items
-rw-r--r-- 1 Walid supergroup 151335822 2026-09-11 16:19 /movies/movies_dataset.csv
C:\>hadoop\bin>

Administrator: Command Prompt
C:\>hadoop\bin>hadoop jar "C:\Users\Walid\Desktop\bda\movieslab\target\movie-mapreduce-1.0-SNAPSHOT.jar" com.example.mo
ies.MovieTop10 /movies /totalBoxOffice
Found 1 items
rw-r--r-- 1 Walid supergroup 151335822 2026-09-11 16:19 /movies/movies_dataset.csv
C:\>hadoop\bin>hadoop jar "C:\Users\Walid\Desktop\bda\movieslab\target\movie-mapreduce-1.0-SNAPSHOT.jar" com.example.mo
ies.MovieTop10 /movies /totalBoxOffice
2026-09-11 16:39:19,812 INFO client.RNProxy: Connecting to ResourceManager at /0.0.0.0:8032
2026-09-11 16:39:20,558 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement
the Tool interface and execute your application with ToolRunner to remedy this.
2026-09-11 16:39:20,574 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/
Walid/.staging/job_1789121862007_0001
2026-09-11 16:39:21,482 INFO InputFileInputFormat: Total input files to process : 1
2026-09-11 16:39:21,466 INFO mapreduce.JobSubmitter: number of splits:2
2026-09-11 16:39:22,030 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1789121862007_0001
2026-09-11 16:39:22,033 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-09-11 16:39:22,221 INFO conf.Configuration: resource-types.xml not found
2026-09-11 16:39:22,221 INFO resource.ResourceUtil: Unable to find 'resource-types.xml'
2026-09-11 16:39:22,865 INFO impl.YarnClientImpl: Submitted application application_1789121862007_0001
2026-09-11 16:39:22,910 INFO mapreduce.Job: The url to track the job: http://DESKTOP-VUHQ7KEL:8088/proxy/application_1789
121862007_0001/
2026-09-11 16:39:22,911 INFO mapreduce.Job: Running job: job_1789121862007_0001
2026-09-11 16:39:32,218 INFO mapreduce.Job: Job job_1789121862007_0001 running in uber mode : false
2026-09-11 16:39:32,220 INFO mapreduce.Job: map 0% reduce 0%
2026-09-11 16:39:40,449 INFO mapreduce.Job: map 50% reduce 0%
2026-09-11 16:39:42,478 INFO mapreduce.Job: map 100% reduce 0%
2026-09-11 16:39:47,549 INFO mapreduce.Job: map 100% reduce 100%
2026-09-11 16:39:48,565 INFO mapreduce.Job: Job job_1789121862007_0001 completed successfully
2026-09-11 16:39:48,635 INFO mapreduce.Job: Counters: 54

Administrator: Command Prompt
C:\>hadoop\bin>hdfs dfs -cat /output/part-r-00000
Scientist picture $1,499,496,720
Among religious night $1,499,467,818
Happy religious serious citizen $1,496,190,333
Free cup day tree $1,495,816,485
Cut $1,495,192,277
Seam fish send people $1,494,807,402
Production five rule $1,493,571,871
Which team deep $1,492,659,744
Serve view $1,489,685,809
Take exist good $1,487,863,886
C:\>hadoop\bin>

```

Figure 4: Hadoop MapReduce execution and output for Top 10 Highest Grossing Movies.

6.2 Total Global Box Office by Genre

Objective

To calculate the total Global_BoxOfficeUSD generated by movies belonging to each genre.

Method

For each valid movie record, the Mapper extracts the genre and global box-office revenue and emits: Genre → Global Box Office Revenue

During the Shuffle and Sort phase, all revenue values belonging to the same genre are grouped together. The Reducer then adds these values to calculate the total global box-office revenue for each genre.

Workflow

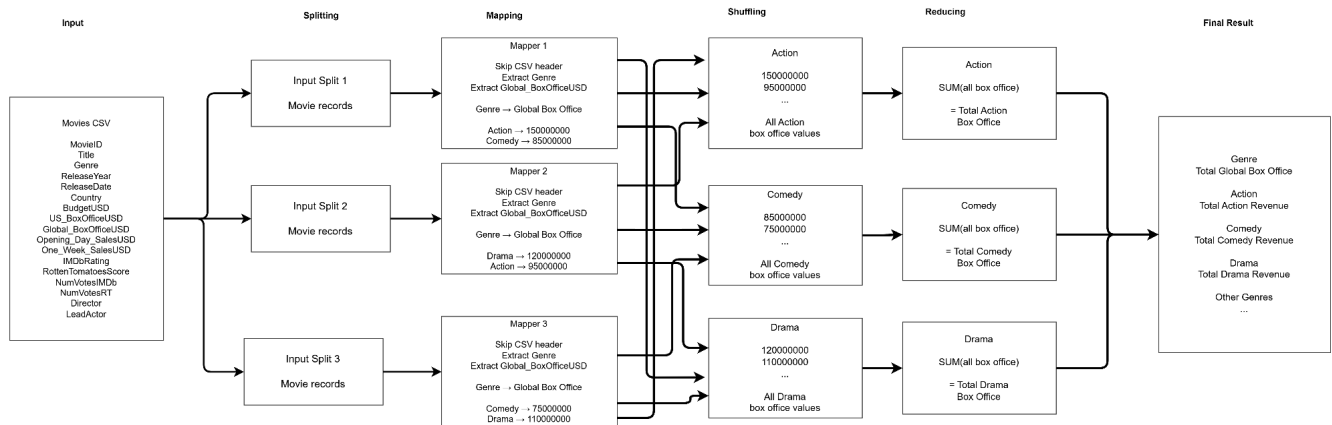


Figure 5: MapReduce Workflow for Total Global Box Office by Genre

Mapper

The main Mapper logic is:

```
String genreValue = fields[2].trim();

double globalBoxOffice =
Double.parseDouble(fields[8].trim());

genre.set(genreValue);
boxOffice.set(globalBoxOffice);

context.write(genre, boxOffice);
```

The Mapper therefore converts each movie into a key-value pair: Genre → Global Box Office Revenue

Movies belonging to the same genre are later grouped together by Hadoop.

Reducer

The Reducer receives all global box-office values for the same genre and calculates their sum:

```
double total = 0.0;

for (DoubleWritable value : values) {
total += value.get();
}

String formattedTotal = String.format("%.2f", total);
```

```
context.write(key, new Text(formattedTotal));
```

The Reducer therefore performs the aggregation: Genre → Total Global Box Office

Driver

The Driver configures and submits the MapReduce job:

```
Job job = Job.getInstance(
    conf,
    "Total Global Box Office by Genre"
);

job.setJarByClass(GenreBoxOffice.class);

job.setMapperClass(GenreMapper.class);
job.setReducerClass(GenreReducer.class);

job.setMapOutputKeyClass(Text.class);
job.setMapOutputValueClass(DoubleWritable.class);

job.setOutputKeyClass(Text.class);
job.setOutputValueClass(Text.class);

FileInputFormat.addInputPath(job, new Path(args[0]));
FileOutputFormat.setOutputPath(job, new Path(args[1]));

System.exit(job.waitForCompletion(true) ? 0 : 1);
```

The Driver connects the input data with the Mapper and Reducer and specifies the input and output HDFS paths. It also defines Text as the key type and DoubleWritable as the intermediate Mapper value type.

Result

Genre	Total Global Box Office
Action	\$4,030,095,779,474.21
Comedy	\$5,439,413,114,727.42
Documentary	\$1,382,233,107,288.92
Drama	\$6,788,589,758,758.33
Horror	\$2,748,348,306,425.94
Romance	\$2,728,887,424,760.39

Sci-Fi	\$1,343,275,863,715.20
Thriller	\$2,745,383,120,770.95

Table 4: Total Global Box Office by Genre Using MapReduce

Observation

Drama has the highest aggregate global box-office revenue among the eight genres, followed by Comedy. Sci-Fi has the lowest aggregate total.

Figure 6: Hadoop MapReduce execution and output for Total Global Box Office by Genre.

7. Pig Latin Analysis

Apache Pig was used for analyses that can be expressed naturally through high-level data-flow operations. The main operators used are LOAD, FILTER, FOREACH, ORDER, LIMIT and STORE.

7.1 Top 10 Global Revenue Movies

Objective

To identify the ten movies with the highest Global_BoxOfficeUSD using Pig Latin.

Method

The script loads the CSV, orders records by Global_BoxOfficeUSD in descending order, limits the result to ten, and projects Title and global revenue. The revenue is cast too long to avoid scientific notation in the output.

```
ordered = ORDER movies BY Global_BoxOfficeUSD DESC;
top10 = LIMIT ordered 10;
result = FOREACH top10 GENERATE
    Title, (long)Global_BoxOfficeUSD AS GlobalRevenue;
STORE result INTO 'top10_global_revenue'
USING PigStorage("");
```

Process Workflow

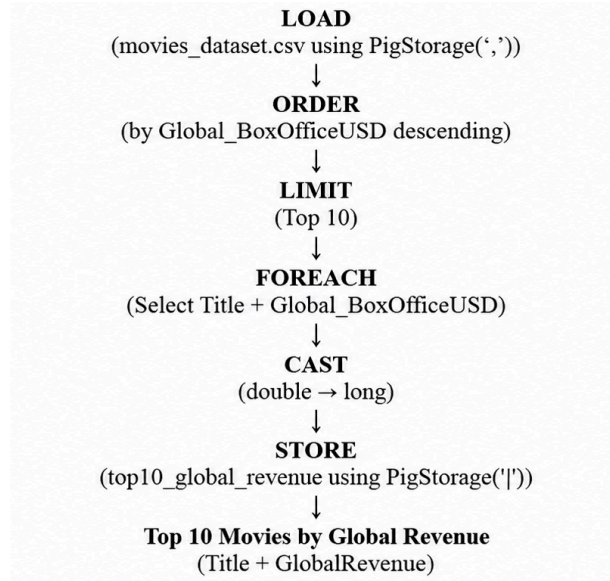


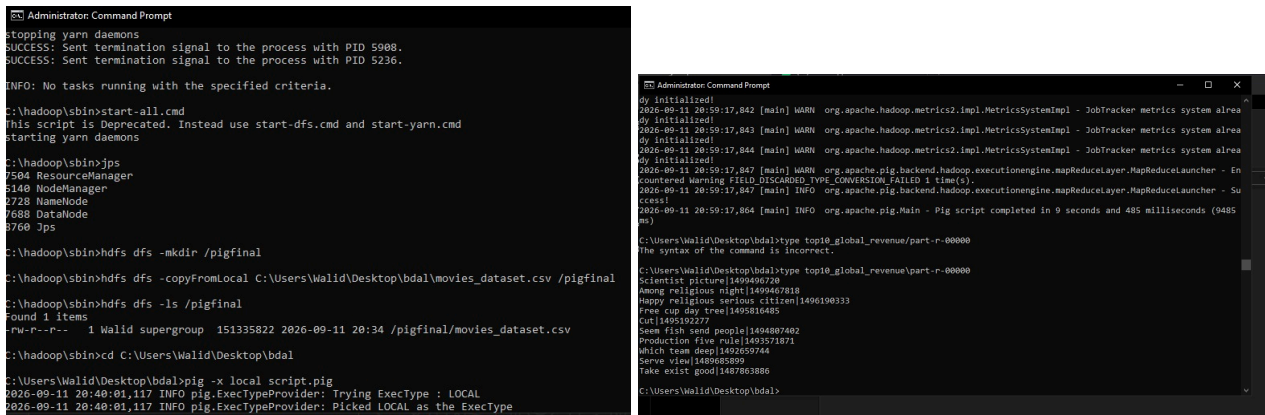
Figure 7: Pig Latin Workflow for Top 10 Global Revenue Movies

Result

Rank	Movie Title	Global Revenue (USD)
1	Scientist picture	1,499,496,720
2	Among religious night	1,499,467,818
3	Happy religious serious citizen	1,496,190,333
4	Free cup day tree	1,495,814,648
5	Cut	1,495,192,277
6	Seem fish send people	1,494,807,402
7	Production five rule	1,493,571,871
8	Which team deep	1,492,659,744
9	Serve view	1,489,685,999
10	Take exist good	1,487,863,886

Table 5: Top 10 Global Revenue Movies Using Pig Latin

Observation: Pig expresses the Top-10 operation directly through `ORDER` and `LIMIT` with considerably less custom algorithmic code.



```
Administrator: Command Prompt
stopping yarn daemons
SUCCESS: Sent termination signal to the process with PID 5908.
SUCCESS: Sent termination signal to the process with PID 5236.
INFO: No tasks running with the specified criteria.

C:\hadoop\sbin>start-all.cmd
This script is deprecated. Instead use start-dfs.cmd and start-yarn.cmd
starting yarn daemons

C:\hadoop\sbin>jps
7504 ResourceManager
5140 NodeManager
5728 NameNode
7688 DataNode
8760 Jps

C:\hadoop\sbin>hdfs dfs -mkdir /pigfinal

C:\hadoop\sbin>hdfs dfs -copyFromLocal C:\Users\Walid\Desktop\bdal\movies_dataset.csv /pigfinal

C:\hadoop\sbin>hdfs dfs -ls /pigfinal
Found 1 items
-rw-r--r-- 1 Walid supergroup 151335822 2026-09-11 20:34 /pigfinal/movies_dataset.csv

C:\hadoop\sbin>cd C:\Users\Walid\Desktop\bdal

C:\Users\Walid\Desktop\bdal>pig -x local script.pig
2026-09-11 20:40:01,117 INFO pig.ExecTypeProvider: Trying ExecType : LOCAL
2026-09-11 20:40:01,117 INFO pig.ExecTypeProvider: Picked LOCAL as the ExecType

Administrator: Command Prompt
dy initialized
2026-09-11 20:59:17,842 [main] WARN org.apache.hadoop.metrics2.impl.MetricsSystemImpl - JobTracker metrics system alrea
dy initialized
2026-09-11 20:59:17,843 [main] WARN org.apache.hadoop.metrics2.impl.MetricsSystemImpl - JobTracker metrics system alrea
dy initialized
2026-09-11 20:59:17,844 [main] WARN org.apache.hadoop.metrics2.impl.MetricsSystemImpl - JobTracker metrics system alrea
dy initialized
2026-09-11 20:59:17,847 [main] WARN org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - En
countered Warning FIELD_DISCARDED_TYPE_CONVERSION_FAILED 1 time(s).
2026-09-11 20:59:17,847 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - Su
ccess!
2026-09-11 20:59:17,864 [main] INFO org.apache.pig.Main - Pig script completed in 9 seconds and 485 milliseconds (9485
ms)
C:\Users\Walid\Desktop\bdal>type top10_global_revenue/part-r-00000
The syntax of the command is incorrect.
C:\Users\Walid\Desktop\bdal>type top10_global_revenue/part-r-00000
Scientist picture|1499496720
Among religious night|1499440718
Happy religious services citizen|1496190333
Free cup day tree|1495816485
Cut|1495192277
Swen fish send people|1494887402
Production five rule|1493571871
Which team deep|1492659744
Serve view|1489685899
Take exist good|1487863886
C:\Users\Walid\Desktop\bdal>
```

Figure 8: Pig Execution and Output for Top Ten Global Revenue

7.2 Movies with Positive ROI

Objective

To calculate the Return on Investment (ROI) for each movie and retain only movies with a positive ROI.

Method

The script uses FOREACH to create the ROI field and FILTER to retain only ROI > 0. The supplied execution processed approximately one million records and successfully stored 889,307 positive-ROI records. The ROI is calculated using: $ROI = ((\text{Global Box Office} - \text{Budget}) / \text{Budget}) \times 100$.

A positive ROI indicates that the recorded global box-office revenue is greater than the recorded production budget.

```
roi_movies = FOREACH movies_no_header GENERATE
    MovieID, Title, Genre, BudgetUSD,
    Global_BoxOfficeUSD,
    ((Global_BoxOfficeUSD - BudgetUSD) / BudgetUSD) * 100
    AS ROI:double;

profitable_movies = FILTER roi_movies BY ROI > 0;

STORE profitable_movies INTO 'positive_roi'
USING PigStorage("");
```

Process Workflow

Process Flow for Profitable Movies ROI Calculation

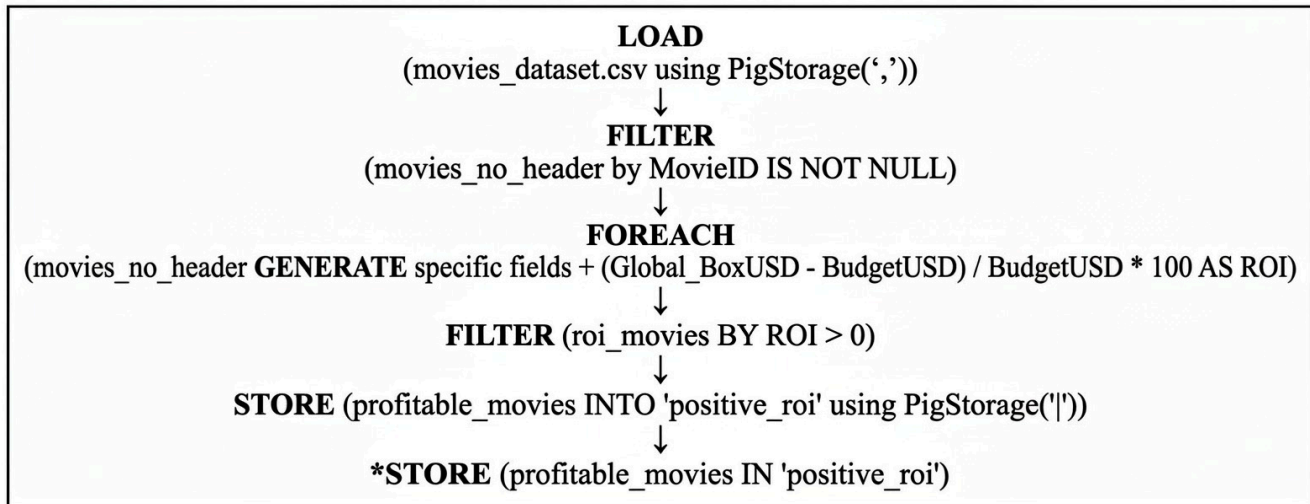


Figure 9: Pig Latin Workflow for Positive ROI Analysis

Result

Rank	Title	Genre	Budget	Global Revenue	ROI
1	Might toward capital	Comedy	\$6,577,427.79	\$15,472,035.66	135.23%
2	He however experience	Comedy	\$1,883,810.10	\$3,637,731.12	93.10%
3	Star responsibility politics	Comedy	\$2,468,079.29	\$7,165,111.24	190.31%
4	Exactly live	Comedy	\$1,447,311.46	\$4,373,820.26	202.20%
5	Focus improve especially	Documentary	\$900,915.86	\$3,113,017.38	245.54%

Table 6: Sample Movies with Positive ROI

Observation

889,307 of the 999,999 data records satisfy $ROI > 0$, corresponding to approximately 88.93%. The output is a filtered profitability dataset rather than a ranked list.

```

Administrator: Command Prompt
starting yarn daemons
C:\Hadoop\bin>ps
6024 Jps
8296 ResourceManager
1452 NodeManager
1740 NameNode
7708 DataNode

C:\Hadoop\bin>hdfs dfs -mkdir /pigfinal
C:\Hadoop\bin>hdfs dfs -copyFromLocal C:\Users\Walid\Desktop\bdal\movies_dataset.csv /pigfinal
C:\Hadoop\bin>hdfs dfs -ls /pigfinal
Found 1 items
-rw-r--r-- 1 Walid supergroup 151335822 2026-09-11 19:23 /pigfinal/movies_dataset.csv

C:\Hadoop\bin>cd C:\Users\Walid\Desktop\bdal
C:\Users\Walid\Desktop\bdal>pig -x local script.pig
2026-09-11 19:30:24,555 INFO pig.ExecTypeProvider: Trying ExecType : LOCAL
2026-09-11 19:30:24,536 INFO pig.ExecTypeProvider: Picked LOCAL as the ExecType
2026-09-11 19:30:24,696 [main] INFO org.apache.pig.Main - Apache Pig version 0.18.0 (r1926484) compiled Jun 16 2025, 10:38:21
2026-09-11 19:30:24,696 [main] INFO org.apache.pig.Main - Logging error messages to: C:\Hadoop\logs\pig_1789133424694.1.log
2026-09-11 19:30:24,712 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - user.name is deprecated. Instead, use mapreduce.job.user.name
2026-09-11 19:30:24,884 [main] INFO org.apache.pig.impl.util.Utils - Default bootstrap file C:\Users\Walid/.pigbootstrap not found

Instead, use mapreduce.job.maps
2026-09-11 19:30:32,151 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapred.reduce.tasks is deprecated. Instead, use mapreduce.job.reduces
2026-09-11 19:30:32,152 [main] WARN org.apache.hadoop.metrics2.impl.MetricsSystemImpl - JobTracker metrics system already initialized!
2026-09-11 19:30:32,175 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - 10% complete
2026-09-11 19:30:32,178 [main] INFO org.apache.pig.tools.pigstats.mapreduce.SimplePigStats - Script Statistics:

HadoopVersion  PigVersion  UserId  StartedAt  FinishedAt  Features
0.2.4  0.18.0  Walid  2026-09-11 19:30:25  2026-09-11 19:30:32  FILTER
Success!

Job Stats (time in seconds):
JobId  Maps  Reduces  MaxMapTime  MinMapTime  AvgMapTime  MedianMapTime  MaxReduceTime  MinReduceTime
job_local1565917024_0001  5  0  n/a  n/a  n/a  n/a  0  0
movies_no_header,profitable_movies,roi_movies  MAP_ONLY  file:///C:/Users/Walid/Desktop/bdal/positive_roi,

Input(s):
Successfully read 1000000 records from: "file:///C:/Users/Walid/Desktop/bdal/movies_dataset.csv"
Output(s):
Successfully stored 889307 records in: "file:///C:/Users/Walid/Desktop/bdal/positive_roi"
Counters:
Total records written : 889307
Total bytes written : 0
  
```

Figure 10: Pig Execution and Output for Positive ROI Movies

8. Comparative Analysis: MapReduce vs Pig

Java MapReduce and Pig Latin were both used to analyze the movie dataset, but they provide different levels of abstraction. Java MapReduce is a lower-level approach in which the Mapper and Reducer classes are implemented explicitly, giving greater control over parsing, algorithms, data structures, and processing logic. In this project, MapReduce was used for tasks such as finding the Top 10 highest-grossing movies using a PriorityQueue and calculating total global box-office revenue by genre using Reducer-based aggregation. In contrast, Pig Latin provides a higher-level data-flow approach where operations such as FILTER, ORDER, LIMIT, FOREACH, and STORE can perform common data-processing tasks with considerably less code. In this project, Pig was used to identify the Top 10 movies by global revenue and calculate and filter movies based on positive ROI. Therefore, MapReduce is more suitable when detailed algorithmic control or custom processing is required, while Pig Latin is more convenient for standard data transformations, filtering, sorting, limiting, and calculated-field operations. Overall, the two approaches demonstrate different ways of performing Big Data analysis on the same movie dataset: MapReduce provides greater programming control, whereas Pig provides simpler and more concise data-flow processing.

9. Results and Discussion

Analysis	Result / Observation
Top 10 Highest Grossing Movies	Ten movies ranked by Global_BoxOfficeUSD using a PriorityQueue-based reducer.
Total Global Box Office by Genre	Eight genres were processed. Drama has the highest aggregate global revenue.
Pig Top 10 Global Revenue	The same Top-10 ranking is expressed using ORDER and LIMIT.
Positive ROI Movies	889,307 records satisfy $ROI > 0$, approximately 88.93% of the supplied data records.

Table 7: Summary of Mapreduce and Pig Analysis

The two Top-10 implementations produce the same ranking, demonstrating that the same analytical problem can be implemented either through a custom MapReduce algorithm or through Pig's declarative data-flow operators.

The genre analysis demonstrates key-based aggregation, while the ROI analysis demonstrates calculated fields followed by conditional filtering.

10. Limitations and Learning Outcomes

10.1 Limitations

- Simple delimiter-based CSV parsing may misinterpret quoted text containing commas.
- The ROI calculation uses only BudgetUSD and Global_BoxOfficeUSD. It should therefore be interpreted as a dataset-level box-office ROI rather than a complete accounting profit measure.

10.2 Learning Outcomes

- HDFS-based storage and Hadoop job execution.
- Mapper and Reducer programming.
- Shuffle and Sort and key-based aggregation.
- Global Top-K processing with PriorityQueue.
- Pig Latin data-flow processing.
- Calculated fields and conditional filtering.
- Use of LOAD, FILTER, FOREACH, ORDER, LIMIT and STORE.
- Comparison of low-level MapReduce and high-level Pig processing.

11. Conclusion

This project demonstrated how Hadoop MapReduce and Apache Pig can be used to analyze a large movie dataset and transform raw records into meaningful information about global revenue, genre performance and profitability. MapReduce provided explicit control over Mapper and Reducer logic. The Top-10 program used a fixed-size PriorityQueue, while the genre program demonstrated the standard key-based aggregation pattern.

Pig provided a concise data-flow approach. ORDER and LIMIT expressed the Top-10 analysis, while FOREACH and FILTER calculated ROI and retained profitable movies. The positive-ROI analysis identified 889,307 records satisfying $ROI > 0$. Overall, the project demonstrates the practical difference between algorithm-oriented Java MapReduce and declarative Pig Latin and provides hands-on experience with large-scale data processing.